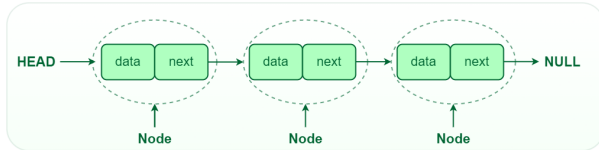


Linked List

1. What is a linked list? Discuss with example. Ans: **Linked Lists:** A chain of nodes, where each node contains a data element and a reference (or pointer) to the next node. Linked lists can be singly linked (each node points to the next) or doubly linked (each node points to both the next and the previous).



Node Structure: A node in a linked list typically consists of two components: **1.Data:** It holds the actual value or data associated with the node. **2.Pointer:** It stores the memory address (reference) of the next node in the sequence.

Head and Tail: The linked list is accessed through the head node, which points to the first node in the list. The last node in the list points to NULL or nullptr, indicating the end of the list. This node is known as the tail node.

2. Explain the main differences between the linked list and linear array? Ans: Linked lists are a fundamental data structure in computer science that consist of a sequence of nodes, where each node contains a value and a reference to the next node in the list.

0.Characteristic: 0.Linked List. 0.Linear Array (Array).

- 1. Structure.** 1. Consists of nodes, each with data and a reference to the next node. 1.Contiguous block of memory with elements indexed by position.
- 2.Dynamic Size.** 2.Dynamic; can easily grow or shrink by adding or removing nodes. 2.Static; size is fixed at the time of declaration.
- 3.Memory Allocation.** 3.Non-contiguous; nodes can be scattered in memory. 3.Contiguous; elements stored sequentially in memory.
- 4. Access Time.** 4.Requires traversal from one node to another. 4. Direct access using index (position).
- 5.Insertions/Deletions.** 5.Efficient for insertions and deletions within the list. 5.Less efficient, especially for middle insertions/deletions.
- 6.Memory Efficiency.** 6.No wasted memory for unused space in the list. 6.May waste memory if not fully utilized.

3. Briefly discuss the terms garbage collection, overflow and Underflow. Ans:

Garbage Collection: Garbage collection is an automatic memory management process in programming languages like Java and C#. It identifies and reclaims memory occupied by objects that are no longer accessible, preventing memory leaks and optimizing memory usage.

Overflow: Overflow occurs when a data type's value exceeds its maximum representable limit. For example, if an integer variable meant to store values between -32,768 and 32,767 exceeds this range, an overflow occurs, often resulting in unexpected behavior or data corruption.

Underflow: Underflow is the opposite of overflow and happens when a data type's value falls below its minimum representable limit. For instance, if a floating-point number intended to store small values becomes too small to be represented accurately, an underflow occurs, potentially leading to loss of precision.

4. One of the advantages of linked lists is the ability to insert data into the list easily. Explain with your own words and figures how to insert data at the beginning, after a given node, at the end and to a sorted list. Ans: Linked lists are a fundamental data structure in computer science that consist of a sequence of nodes, where each node contains a value and a reference to the next node in the list. One of the advantages of linked lists is their ability to easily insert data at various positions within the list.

Inserting Data at the Beginning:

To insert data at the beginning of a linked list, a new node is created with the desired value. The next pointer of this new node is then set to point to the current head of the list. Finally, the head pointer is updated to point to the newly inserted node.

Before insertion: Head -> Node1 -> Node2 -> ... -> NodeN

After insertion: NewNode -> Head -> Node1 -> Node2 -> ... -> NodeN

Inserting Data after a Given Node: To insert data after a specific node in a linked list, we need to locate that node first. Once we have found the desired node, we create a new node with the desired value and update its next pointer to point to the next node in the original list. Then, we update the next pointer of the given node to point to our newly inserted node.

Before insertion: Head -> Node1 -> GivenNode -> Node2 -> ... -> NodeN

After insertion: Head -> Node1 -> GivenNode -> NewNode -> Node2 -> ... -> NodeN

Inserting Data at the End: To insert data at the end of a linked list, we need to traverse through the entire list until we reach the last node. Once we have reached the last node, we create a new node with the desired value and set its next pointer to null. Then, we update the next pointer of the current last node to point to our newly inserted node.

Before insertion: Head -> Node1 -> Node2 -> ... -> NodeN -> LastNode

After insertion: Head -> Node1 -> Node2 -> ... -> NodeN -> LastNode -> NewNode

Inserting Data into a Sorted List: Inserting data into a sorted linked list requires finding the correct position for the new node based on its value. We start by traversing the list from the head until we find a node whose value is greater than the value we want to insert. Once we have found the appropriate position, we create a new node with the desired value and update its next pointer to point to the next node in the original list. Then, we update the next pointer of the previous node to point to our newly inserted node.

Before insertion: Head -> Node1 -> Node2 -> ... -> PrevNode -> NextNode -> ... -> LastNode

After insertion: Head -> Node1 -> Node2 -> ... -> PrevNode -> NewNode -> NextNode -> ... -> LastNode

5. Can binary search algorithm be employed on data stored in linked list? Justify your answer. Ans: Yes, the binary search algorithm can be employed on data stored in a linked list, but with certain limitations and modifications. Binary search is a divide-and-conquer algorithm that requires random access to elements, which is not possible in a linked list. However, by using a modified approach called "binary search on a sorted linked list," it is still possible to perform a binary search.

In this modified approach, instead of accessing elements directly through indexing, we traverse the linked list by comparing the target value with the middle element of the current sublist. Based on the comparison, we can determine whether to continue searching in the left or right half of the sublist. By repeatedly dividing the sublist in half, we can efficiently locate the target element.

However, it is important to note that binary search on a linked list has some drawbacks compared to an array-based implementation. The time complexity increases from $O(\log n)$ to $O(n)$, as we need to traverse the list linearly. Additionally, random access is not possible, affecting efficiency.

Linked List

13. Sort Elements of a Linked List:

We will use a simple sorting algorithm, Bubble Sort, to sort the elements of a linked list in ascending order below.

1. Make the head as the current node and create another node index for later use. **2.** If head is null, return. **3.** Else, run a loop till the last node (i.e. NULL).

4. In each iteration, follow the following step 5-6. **5.** Store the next node of current in index. **6.** Check if the data of the current node is greater than the next node. If it is greater, swap current and index.

// Sort the linked list

```
void sortLinkedList(struct Node** head_ref) {
    struct Node *current = *head_ref, *index = NULL;
    int temp;
```

```
    if (head_ref == NULL) {
        return;
    } else {
        while (current != NULL) {
            // index points to the node next to current
            index = current->next;
```

```
            while (index != NULL) {
                if (current->data > index->data) {
                    temp = current->data;
                    current->data = index->data;
                    index->data = temp;
                }
                index = index->next;
            }
            current = current->next;
        }
    }
}
```

14. How to find an element on the first over array and linked list.

Ans: Array supports Random Access, which means elements can be accessed directly using their index, like arr[0] for 1st element, arr[6] for 7th element etc. Hence, accessing elements in an array is fast with a constant time complexity of $O(1)$.

Linked List supports Sequential Access, which means to access any element/node in a linked list, we have to sequentially traverse the complete linked list, upto that element. To access nth element of a linked list, time complexity is $O(n)$.

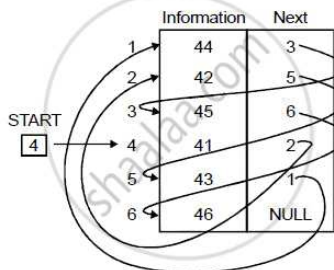
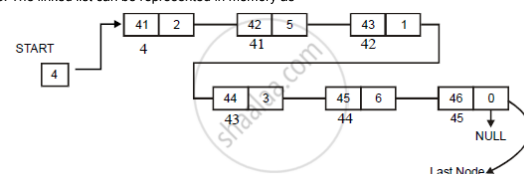
15. How Linked List are Represented in Memory?

Ans:

1. Linked lists can be represented in memory by using two arrays respectively known as INFO and LINK, such that INFO[K] and LINK[K] contains information of element and next node address respectively.

2. The list also requires a variable 'Name' or 'Start', which contains address of first node. Pointer field of last node denoted by NULL which indicates the end of list. e.g., Consider a linked list given below :

3. The linked list can be represented in memory as-



Linked List

16. what is header linked list?

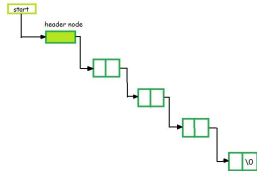
Ans:A header node is a special node that is found at the *beginning* of the list. A list that contains this type of node, is called the header-linked list. This type of list is useful when information other than that found in each node is needed.

17. Types of Header Linked List.(Grounded Header,Circular Header)

Ans:

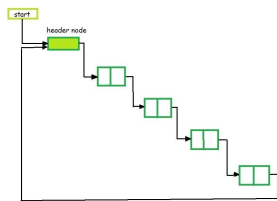
1.Grounded Header Linked List

It is a list whose last node contains the NULL pointer. In the header linked list the start pointer always points to the header node. start -> next = NULL indicates that the grounded header linked list is empty. The operations that are possible on this type of linked list are Insertion, Deletion, and Traversing.



2. Circular Header Linked List

A list in which last node points back to the header node is called circular linked list. The chains do not indicate first or last nodes. In this case, external pointers provide a frame of reference because last node of a circular linked list does not contain the NULL pointer. The possible operations on this type of linked list are Insertion, Deletion and Traversing.



18. What is a doubly linked list, and how is it different from a singly linked list?Ans: A two-way linked list, also known as a doubly linked list, is a variation of a linked list where each node has two pointers: one pointing to the next node and another pointing to the previous node. This bidirectional linkage allows for more flexible traversal in both directions, from the head to the tail and vice versa.

0.Aspect 0.Singly Linked List 0.Doubly Linked List

- 1.Bidirectional Pointers 1.No 1.Yes
- 2.Traversal Efficiency 2.Efficient in one direction. 2.Efficient in both directions.
- 3.Insertions and Deletions. 3.Efficient at the beginning and middle. 3.Efficient for insertions and deletions throughout.
- 4.Memory Overhead.4.Lower (single pointer per node). 4.Slightly higher (two pointers per node).
5. Complexity. 5.Simpler to implement. 5.Slightly more complex to implement.
- 6.Typical Use Cases. 6.Simple linked structures. 6.When bidirectional traversal or frequent insertions/deletions are required.

Hashing

1.what is hash function?Ans:A Hash Function is a function that converts a given numeric or alphanumeric key to a small practical integer value. The mapped integer value is used as an index in the hash table. In simple terms, a hash function maps a significant number or string to a small integer that can be used as the index in the hash table.

2.what can be the techniques to avoid collusion?Ans:To avoid collisions in a hash table:

1. Use a Good Hash Function: Select a quality hash function that distributes keys evenly.
2. Open Addressing: Handle collisions by placing the item in the next available slot.
3. Separate Chaining: Use linked lists or other data structures to store multiple items in the same hash slot.
4. Dynamic Resizing: Rehash the table when it reaches a certain load factor to maintain a balanced table.
5. Double Hashing: Combine multiple hash functions to find the next available slot for collisions.
6. Robin Hood Hashing: Reorganize elements in case of collisions to minimize the difference in probe lengths.

These techniques reduce the likelihood of collisions and ensure efficient hash table performance.

3.briefly discuss different hash collision resolution techniques.Ans:Hash collision resolution techniques include:1. Open Addressing: Linear probing, quadratic probing, and double hashing search for the next available slot when collisions occur.2. Separate Chaining: Slots hold linked lists or data structures to store colliding keys.3. Robin Hood Hashing: Reorganizes elements during collisions to minimize differences in probe lengths.4. Cuckoo Hashing: Uses multiple hash functions and two tables, swapping items between tables upon collisions.5. Coalesced Hashing: Combines open addressing and chaining, allowing limited collisions before using chaining.6. Double Hashing: Utilizes a second hash function to determine probe steps, reducing clustering.7. Linear/Extendible Hashing: Dynamically adjusts the hash table's size to minimize collisions.Each technique addresses collisions differently, catering to various use cases based on key distribution and performance requirements.

4.Open hashing or chaining.Ans:Open hashing or more widely known as chaining is one of the simplest approaches to avoid collision in hash tables. In open hashing, each hash table slot, also known as a bucket, contains a linked list of elements that hash to the same slot. The term chaining is used to describe the process because the linked list is used, which is like a chain of elements.

5.Closed Hashing Ans:Like open hashing, closed hashing is also a technique used for collision resolution in hash tables. Unlike open hashing, where collisions are resolved by chaining elements in separate chains, closed hashing aims to store all elements directly within the hash table itself without the use of additional data structures like linked lists.

6.Linear Probing:In linear probing, if a collision occurs and a slot is already occupied, the algorithm linearly searches for the next available slot in a sequential manner. It checks the next slot and continues checking subsequent slots until it finds an empty slot.

7.Quadratic Probing:In quadratic probing, instead of probing the next slot linearly, the algorithm uses a quadratic function to determine the next probe position. The probing sequence follows a quadratic pattern until an empty slot is found.

8.Double hashing:utilizes two hash functions to determine the probe sequence. When a collision occurs, it applies the second hash function to calculate an offset, which is then used to find the next slot to probe.

Linked List

6.What are the advantages of Linked List over array?

Ans:

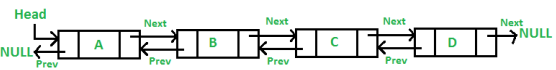
- 0.Advantage. 0.Linked List 0. Array
- 1.Dynamic Size. 1.Can grow or shrink dynamically.. 1.Has a fixed size determined at creation..
- 2.Efficient Insertions/Deletions. 2.Efficient for inserting/deleting elements, especially in the middle.
- 2.Insertions/deletions may require shifting, less efficient.
- 3.Memory Efficiency. 3.Uses memory proportional to the number of elements. 3. May waste memory due to preallocated space.
- 4.No Need for Preallocation. 4.No need to preallocate memory for a maximum size.. 4.Requires preallocation of a fixed size.
- 5.No Wasted Memory. 5.No wasted memory for unused space. 5.May have unused or uninitialized elements.
6. Flexibility. 6.Suited for dynamic data structures. 6.Fixed size can limit flexibility.

7. What is two way lists(doubly linked list)?Ans: A two-way linked list, also known as a doubly linked list, is a variation of a linked list where each node has two pointers: one pointing to the next node and another pointing to the previous node. This bidirectional linkage allows for more flexible traversal in both directions, from the head to the tail and vice versa.

8.Importance of two way lists(doubly linked list)? Explain with Schematic diagram.

Ans: Importance of Two-Way Linked Lists:

Bidirectional Traversal: Doubly linked lists allow efficient traversal in both forward and backward directions, making them valuable for tasks like reverse traversal.**Insertions and Deletions:** They are efficient for inserting and deleting nodes at both the beginning and end of the list, as well as within the list.**Applications:** Doubly linked lists are used in various applications, such as implementing undo/redo functionality in software, navigating browser history, and managing data structures like deque (double-ended queue).



9. Advantage and Disadvantage of Linked List.

Ans:

- 0.Aspect 0.Advantage 0.Disadvantage.
- 1.Dynamic Size. 1.Easily accommodates a varying number of elements.. 1.Does not offer constant-time access to elements.
- 2.Efficient Insertions. 2.Efficient for inserting and deleting elements. 2.Inefficient for random access, O(n) for traversal.
- 3.Low Memory Overhead. 3.Low memory overhead per node. 3.Requires extra memory for storing references.
- 3.Simple Implementation. 4.Conceptually simpler to implement. 4.May not be the best choice for complex operations.

10. Application of Linked List.Ans:1.Linked Lists are used to implement stacks and queues.2.It is used for the various representations of trees and graphs.3.It is used in dynamic memory allocation(linked list of free blocks).4.It is used for representing sparse matrices.5.It is used for the manipulation of polynomials.6.It is also used for performing arithmetic operations on long integers.7.It is used for finding paths in networks.8.In operating systems, they can be used in Memory management, process scheduling and file system.9.Linked lists can be used to improve the performance of algorithms that need to frequently insert or delete items from large collections of data.10.Implementing algorithms such as the LRU cache, which uses a linked list to keep track of the most recently used items in a cache.

11. Delete Data or Node or element from Linked List in the beginning, middle and last.

Ans:

1. Delete from beginning

- Point head to the second node

head = head->next;

2. Delete from end

- Traverse to second last element
- Change its next pointer to null

```

struct Node* temp = head;
while(temp->next->next!=NULL){
    temp = temp->next;
}

```

temp->next = NULL;

3. Delete from middle

- Traverse to element before the element to be deleted
- Change next pointers to exclude the node from the chain

```

for(int i=2; i< position; i++) {
    if(temp->next!=NULL) {
        temp = temp->next;
    }
}
temp->next = temp->next->next;

```

12. Search an Element on a Linked List.

Ans:You can search an element on a linked list using a loop using the following steps. We are finding item on a linked list.1. Make head as the current node.2. Run a loop until the current node is NULL because the last element points to NULL.3. In each iteration, check if the key of the node is equal to item. If it the key matches the item, return true otherwise return false.

```

// Search a node
bool searchNode(struct Node** head_ref, int key) {
    struct Node* current = *head_ref;

```

```

    while (current != NULL) {
        if (current->data == key) return true;
        current = current->next;
    }
    return false;
}

```

Stack and queue

1. Define stack.Ans:

Stack: A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed. Think of it as a stack of plates where you can only add or remove plates from the top.

2. Explain the usage of stack in recursive algorithm implementation.

Ans:Usage in Recursive Algorithm Implementation:

- Stacks are crucial in implementing recursive algorithms, which are algorithms that call themselves to solve a problem.
- When a function in a recursive algorithm is called, its local variables and execution context are pushed onto the stack.
- Each recursive call creates a new stack frame, allowing the algorithm to remember its state at different levels of recursion.
- When the base case of the recursion is reached, the stack starts popping frames, effectively "unwinding" the recursion and allowing the algorithm to compute the final result by combining the results from each level.
- Without a stack, recursive algorithms would not be able to keep track of multiple levels of recursion and could lead to inefficient or incorrect solutions.

3. Write down a routine to insert an element onto a queue.Ans:To insert an element onto a queue, you typically perform the following steps:

Create the Element: Create the element (or data) that you want to insert into the queue.

Allocate Memory (if needed): If you're using a dynamically allocated data structure (e.g., linked list for the queue), allocate memory for the new element.

Set the Value: Assign the value or data to the newly created element.

Update Pointers (if needed): If you're using a linked list to implement the queue, you'll need to update pointers to link the new element to the existing elements in the queue.

4.What is a queue.Ans: A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle, which means that the element that is added first to the queue will be the first one to be removed.

5. how Queue different from a stack and how is it implemented?Ans:

0.Aspect. 0.Queue. 0.Stack.

- Data Structure. 1.Linear data structure. 1.Linear data structure.
- Ordering Principle. 2.FIFO (First-In-First-Out). 2.LIFO (Last-In-First-Out).
- Main Operations. 3.Enqueue (add to the back), Dequeue (remove from the front). 3.Push (add to the top), Pop (remove from the top).
- Typical Use Cases. 4.Simulates real-world queues, such as a line of people waiting for a service. 4.Used for tracking function calls, backtracking, undo functionality, etc.
- Implementation. 5.Can be implemented using arrays or linked lists. 5.Can be implemented using arrays or linked lists.
- Time Complexity. 6.Enqueue: $O(1)$, Dequeue: $O(1)$. 6.Push: $O(1)$, Pop: $O(1)$.
- Example. 7.Queue for a printer job queue. 7.Stack for function call tracking.

6. How to implement Queue using Array?Ans:

- To implement a queue using an array,
 - create an array arr of size n and
 - take two variables front and rear both of which will be initialized to 0 which means the queue is currently empty.
- Element**
 - rear is the index up to which the elements are stored in the array and
 - front is the index of the first element of the array.

7. Implementation of Stack.Ans:A stack can be implemented using an array or a linked list. In an array-based implementation, the push operation is implemented by incrementing the index of the top element and storing the new element at that index. The pop operation is implemented by decrementing the index of the top element and returning the value stored at that index. In a linked list-based implementation, the push operation is implemented by creating a new node with the new element and setting the next pointer of the current top node to the new node. The pop operation is implemented by setting the next pointer of the current top node to the next node and returning the value of the current top node.

8. Write the steps involved in the insertion and deletion of an element in a stack.Ans:Here are the simplified steps for inserting (pushing) and deleting (popping) an element in a stack:

Insertion (Push) in a Stack:

Prepare the Element: Get the element you want to add to the stack.

Find the Top: Identify the top of the stack (the last element added).

Place the Element: Put the new element on top of the stack.

Update the Top: Update the top to point to the newly added element.

Deletion (Pop) from a Stack:

Check if the Stack is Empty: Before popping, make sure the stack is not empty.

Access the Top Element: If the stack is not empty, take the element from the top.

Remove the Top Element: Remove the top element from the stack.

Update the Top: Update the top to point to the next element below (if any).

These steps ensure that elements are added and removed in a Last-In, First-Out (LIFO) order, similar to a stack of plates where you add and remove plates from the top.

8. Write the steps involved in the insertion and deletion of an element in a queue.Ans:

Insertion (Enqueue):Check if the Queue is Full: Before inserting an element, check if the queue is already full to avoid overflow. You typically compare the current size of the queue to its maximum capacity.

Increment Rear Pointer: If the queue is not full, increment the rear pointer. In a circular queue, you may need to use modulo arithmetic to wrap around to the beginning if the rear pointer reaches the end of the underlying data structure (e.g., an array).

Insert the Element: Place the new element in the position indicated by the rear pointer.

Update Queue Size: Increment the size of the queue to reflect the newly added element.

Deletion (Dequeue):Check if the Queue is Empty: Before dequeuing an element, check if the queue is empty to avoid underflow. You typically compare the current size of the queue to zero.

Retrieve the Element: Get the element at the front of the queue. This is the element that will be removed.

Increment Front Pointer: Increment the front pointer to point to the next element in the queue. In a circular queue, you may need to use modulo arithmetic to wrap around if the front pointer reaches the end.

Update Queue Size: Decrement the size of the queue to reflect the removal of the element.

Optional: Return the Removed Element: Depending on your implementation, you may want to return the element that was dequeued as the result of the operation.

9.What is polish notation?What are the benefits of polish notation?Ans:Polish notation, also known as prefix notation, is a mathematical notation in which every operator follows all of its operands. This is in contrast to infix notation, which is the more common notation used in mathematics, where operators are placed between their operands.

Polish notation eliminates ambiguity, simplifies parsing, and offers a compact, consistent, and efficient way to represent and evaluate mathematical expressions, making it suitable for programming languages and stack-based systems.

Tree

1. Root

In a tree data structure, the first node is called as Root Node. Every tree must have root node. Wencan say that root node is the origin of tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree. In above tree, A is a Root node.

2. Edge

In a tree data structure, the connecting link between any two nodes is called as EDGE. In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.

3. Parent

In a tree data structure, the node which is predecessor of any node is called as PARENT NODE. In simple words, the node which has branch from it to any other node is called as parent node. Parent node can also be defined as "The node which has child / children". e.g., Parent (A,B,C,D).

4. Child

In a tree data structure, the node which is descendant of any node is called as CHILD Node. In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes. e.g., Children of D are (H, I, J).

5. Siblings

In a tree data structure, nodes which belong to same Parent are called as SIBLINGS. In simple words, the nodes with same parent are called as Sibling nodes. Ex: Siblings (B,C, D).

6. Leaf

In a tree data structure, the node which does not have a child (or) node with degree zero is called as LEAF Node. In simple words, a leaf is a node with no child.

In a tree data structure, the leaf nodes are also called as External Nodes. External node is also a node with no child. In a tree, leaf node is also called as "Terminal" node. Ex: (K,L,F,G,M,I,J).

7. Degree In a tree data structure, the total number of children of a node (or) number of subtrees of a node is called as DEGREE of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as 'Degree of Tree'.

8. Level

In a tree data structure, the root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on... In simple words, in a tree each step from top to bottom is called as a Level and the Level count starts with '0' and incremented by one at each level (Step). Some authors start root level with 1.

9. Height

In a tree data structure, the total number of edges from leaf node to a particular node in the longest path is called as HEIGHT of that Node. In a tree, height of the root node is said to be height of the tree. In a tree, height of all leaf nodes is '0'.

10. Depth

In a tree data structure, the total number of edges from root node to a particular node is called as DEPTH of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be Depth of the tree. In simple words, the highest depth of any leaf node in a tree is said to be depth of that tree. In a tree, depth of the root node is '0'.

11. Path In a tree data structure, the sequence of Nodes and Edges from one node to another node is called as PATH between that two Nodes. Length of a Path is total number of nodes in that path. In below example the path A - B - E - J has length 4.

12. Sub Tree In a tree data structure, each child from a node forms a subtree recursively. Every child node will form a subtree on its parent node.

13. ancestor:An ancestor of a node in a binary tree is any node that lies on the path from the root node to that particular node. Ancestor nodes are higher in the tree hierarchy and can be found by tracing the path from the root to the target node.

14. Binary search tree inserting. The very first insertion creates the tree. Afterwards, whenever an element is to be inserted. First locate its proper location. Start search from root node then if data is less than key value, search empty location in left sub tree and insert the data. Otherwise search empty location in right sub tree and insert the data.

In a binary search tree, the insertion operation is performed with $O(\log n)$ time complexity. In binary search tree, new node is always inserted as a leaf node. The insertion operation is performed as follows. **Step 1:** Create a newNode with given value and set its left and right to NULL. **Step 2:** Check whether tree is Empty. **Step 3:** If the tree is Empty, then set root to newNode. **Step 4:** If the tree is Not Empty, then check whether value of newNode is smaller or larger than the node (here it is root node). **Step 5:** If newNode is smaller than or equal to the node, then move to its left child. If newNode is larger than the node, then move to its right child. **Step 6:** Repeat the above step until we reach a node (e.i., reach to NULL) where search terminates. **Step 7:** After reaching a last node, then insert the newNode as left child if newNode is smaller or equal to that node else insert it as right child.



15. Complete Binary Tree

In a binary tree, every node can have a maximum of two children. But in strictly binary tree, every node should have exactly two children or none and in complete binary tree all the nodes must have exactly two children and at every level of complete binary tree there must be 2 level number of nodes. For example at level 2 there must be $2^2 = 4$ nodes and at level 3 there must be $2^3 = 8$ nodes.

16. link representation of a binary tree in memory?Ans:

Binary trees in linked representation are stored in the memory as linked lists. These lists have nodes that aren't stored at adjacent or neighboring memory locations and are linked to each other through the parent-child relationship associated with trees.

In this representation, each node has three different parts –

- pointer that points towards the right node,
- pointer that points towards the left node,
- data element.

This is the more common representation. All binary trees consist of a root pointer that points in the direction of the root node. When you see a root node pointing towards null or 0, you should know that you are dealing with an empty binary tree. The right and left pointers store the address of the right and left children of the tree.

Tree

17. Binary search tree .

A Binary Search Tree (BST) is a binary tree data structure in which each node has at most two children, referred to as the left child and the right child.

18.how can binary search tree overcome the disadvantage of array and linked list

Binary Search Trees (BSTs) offer efficient search ($O(\log n)$) and dynamic sizing like linked lists, overcoming array limitations. Unlike arrays, they allow ordered storage. However, BSTs can suffer from worst-case performance and higher memory overhead due to pointers. Self-balancing BSTs like AVL and Red-Black trees mitigate worst-case scenarios, maintaining efficient operations. Overall, BSTs provide a balance between search efficiency, dynamic sizing, and ordered storage, making them advantageous in many scenarios.

19.what is an extended tree

An "extended tree" is not a standard term in computer science or data structures. It doesn't have a widely recognized or defined meaning in the context of trees or tree data structures.

20.illustrate similar and copies with example

Similarity: In the world of music, two songs with similar melodies or rhythms, like "Let It Be" by The Beatles and "No Woman, No Cry" by Bob Marley, share a musical similarity due to their soothing, ballad-like qualities.

Copies: In literature, a copy of a book might be an identical reprint of the original, preserving the same text and content as the first edition. However, a revised edition of a book is a modified copy that includes updates, corrections, or additional material, like an updated version of a classic novel with an author's commentary.

21.Preorder Traversal,Postorder Traversal,inorder traversal.

Preorder Traversal:

- 1.Visit the root.
- 2.Traverse the left subtree, i.e., call Preorder(left->subtree)
- 3.Traverse the right subtree, i.e., call Preorder(right->subtree)

Postorder Traversal:

- 1.Traverse the left subtree, i.e., call Postorder(left->subtree)
- 2.Traverse the right subtree, i.e., call Postorder(right->subtree)
- 3.Visit the root

inorder traversal:

- 1.Traverse the left subtree, i.e., call Inorder(left->subtree)
- 2.Visit the root.
- 3.Traverse the right subtree, i.e., call Inorder(right->subtree)

21. advantage of binary search tree.

Advantages of Binary Search Tree:

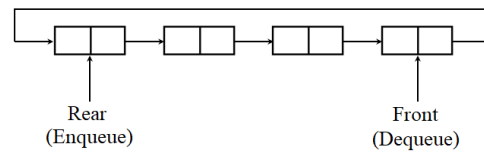
1. BST is fast in insertion and deletion when balanced. ...
2. BST is also for fast searching, with a time complexity of $O(\log n)$ for most operations.
3. BST is efficient. ...
4. We can also do range queries – find keys between N and M ($N \leq M$).
5. BST code is simple as compared to other data structures.

Stack and queue

10.Circular Queue. Ans:A circular queue permits better memory utilization than a simple queue when the queue has a fixed size.

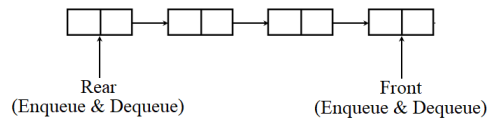
In this queue, the last node points to the first node and creates a circular connection. Thus, it allows us to insert an item at the first node of the queue when the last node is full and the first node is free.

It's also called a ring buffer:



11.Priority Queue. Ans:A priority queue is a special kind of queue in which each item has a predefined priority of service. In this queue, the enqueue operation takes place at the rear in the order of arrival of the items, while the dequeue operation takes place at the front based on the priority of the items.

12. Double-Ended Queue (Deque). Ans:A deque is also a special type of queue. In this queue, the enqueue and dequeue operations take place at both front and rear. That means, we can insert an item at both the ends and can remove an item from both the ends. Thus, it may or may not adhere to the FIFO order:



13. What is recursion. Ans:Recursion is defined as a process which calls itself directly or indirectly and the corresponding function is called a recursive function.

14. explain the use of recursion. Ans:Recursion is used in many fields of computer science and mathematics, which includes: **1.Searching and sorting algorithms:** Recursive algorithms are used to search and sort data structures like trees and graphs. **2.Mathematical calculations:** Recursive algorithms are used to solve problems such as factorial, Fibonacci sequence, etc. **3.Compiler design:** Recursion is used in the design of compilers to parse and analyze programming languages. **4.Graphics:** many computer graphics algorithms, such as fractals and the Mandelbrot set, use recursion to generate complex patterns. **5.Artificial intelligence:** recursive neural networks are used in natural language processing, computer vision, and other AI applications.

Regular

1. Define data structure. Why data Structure is necessary? **Ans:** A data structure is a fundamental concept in computer science, representing a systematic way to organize and store data in a computer's memory. It defines the format and arrangement of data elements, allowing for efficient access, retrieval, and manipulation of information. Data structures serve as building blocks for designing algorithms and solving various computational problems. **Data structures are necessary for several reasons:** **Efficiency:** They enable the optimization of algorithms, ensuring that operations on data can be performed quickly and with minimal resource consumption. **Abstraction:** Data structures abstract the complexities of data management, providing a clear interface for programmers to work with, irrespective of the underlying implementation details. **Reusability:** Well-designed data structures can be reused across different applications, saving time and effort in software development. **Scalability:** They can adapt to handle large datasets and dynamic changes, making them crucial for robust and adaptable software systems. In summary, data structures are essential tools in computer science, facilitating efficient data management, algorithm design, and problem-solving.

2. What do you mean by Linear data Structure and Nonlinear data structure? Give an example.

Ans: Linear data structures and nonlinear data structures are two fundamental categories of data structures in computer science, each with distinct characteristics and use cases.

Linear Data Structure: Linear data structures organize data elements sequentially, with each element connected to the one before and after it. Elements are accessed in a linear, one-dimensional fashion.

Examples of linear data structures include:

a. Arrays: A collection of elements stored in contiguous memory locations. Elements are accessed by their index. For example, an integer array `arr[5]` contains five elements accessed as `arr[0]`, `arr[1]`, ..., `arr[4]`. **b.**

Linked Lists: A chain of nodes, where each node contains a data element and a reference (or pointer) to the next node. Linked lists can be singly linked (each node points to the next) or doubly linked (each node points to both the next and the previous).

Or,

A Linked lists are a fundamental data structure in computer science that consist of a sequence of nodes, where each node contains a value and a reference to the next node in the list.

c. Stacks: A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle, meaning that the last element added to the stack is the first one to be removed. Think of it as a stack of plates where you can only add or remove plates from the top. **d. Queues:** A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle, which means that the element that is added first to the queue will be the first one to be removed.

Nonlinear Data Structure: Nonlinear data structures do not have a sequential arrangement of elements. Elements can be interconnected in various ways, creating complex relationships. Examples include: **a. Trees:** Hierarchical structures where each element (node) has one parent and zero or more children. Common tree types include binary trees, binary search trees, and balanced trees like AVL and Red-Black trees. **b. Graphs:** Collections of nodes (vertices) connected by edges. Graphs can be directed (edges have a direction) or undirected. They are used to represent complex relationships, networks, and various real-world scenarios.

3. How can you differ linear data structures from non-linear data structures? **Ans:** Linear data structures and non-linear data structures differ in how they organize and connect data elements. **Linear Data Structures:** These structures organize data elements sequentially, with each element having a unique predecessor and successor. Elements are accessed in a linear, one-dimensional manner. Examples include arrays, linked lists, stacks, and queues. Linear structures are suitable for tasks where data follows a straightforward order, such as ordered lists or simple data storage. **Non-linear Data Structures:** Non-linear structures do not have a sequential arrangement. Elements are interconnected in complex ways, forming hierarchical or interconnected relationships. Examples include trees and graphs. Non-linear structures are used when data exhibits complex relationships, like hierarchical data, networks, and intricate dependencies among elements.

4. What is the benefit of binary search over linear search? **Ans:**

0. Aspect **0. Binary Search.** **0. Linear Search**

1. Time Complexity. **1. $O(\log n)$ in the worst case.** **1. $O(n)$ in the worst case.**

2. Efficiency. **2. Highly efficient for sorted data.** **2. Linearly searches through data.**

3. Comparison. **3. Divides search space in half with each comparison.** **3. Performs linear comparisons.**

4. Sorted Data Requirement. **4. Requires data to be sorted.** **4. Works with unsorted data as well.**

5. Use Cases. **5. Ideal for large datasets where quick retrieval is essential.** **5. Suitable for small lists or unsorted data.**

6. Example. **6. Searching in a phone directory.** **6. Scanning through an unsorted list.**

5. Briefly discuss the terms garbage collection, overflow and Underflow. **Ans:** **Garbage Collection:**

Garbage collection is an automatic memory management process in programming languages like Java and C#. It identifies and reclaims memory occupied by objects that are no longer accessible, preventing memory leaks and optimizing memory usage. **Overflow:** Overflow occurs when a data type's value exceeds its maximum representable limit. For example, if an integer variable meant to store values between -32,768 and 32,767 exceeds this range, an overflow occurs, often resulting in unexpected behavior or data corruption. **Underflow:** Underflow is the opposite of overflow and happens when a data type's value falls below its minimum representable limit. For instance, if a floating-point number intended to store small values becomes too small to be represented accurately, an underflow occurs, potentially leading to loss of precision.

6. What is 2D array? How can you represent 2D array in memory? **Ans:** A 2D array, also known as a two-dimensional array, is a data structure that organizes data elements in a grid or matrix-like format with rows and columns. It's used to represent tables, matrices, or grids of values in programming. Each element in a 2D array is uniquely identified by its row and column indices.

In memory, a 2D array is typically represented as a contiguous block of memory, with elements arranged in rows and columns. The elements are stored in a sequential manner, row by row or column by column, depending on the programming language and memory layout. The memory is organized such that each row is stored consecutively, ensuring efficient access to individual elements through row and column indexing.

7. How are the elements of a 2D array stored in the memory? **Ans:**

In memory, the elements of a 2D array are stored in a contiguous block, organized in a row-major or column-major fashion, depending on the programming language and memory layout.

Row-major order: In this arrangement, the elements of each row are stored consecutively in memory. This means that elements within the same row are adjacent in memory, while elements in different rows are separated. Accessing elements by row is efficient, as they are stored together.

Column-major order: Here, elements of each column are stored sequentially. Elements within the same column are adjacent in memory, simplifying column-wise access.

The choice of storage order can affect access performance, especially when traversing the array. Most programming languages, like C and C++, use row-major order by default, while others, like Fortran, use column-major order. The choice depends on the language's conventions and the specific requirements of the application.

8. What is sparse matrix? What is the difference between triangular matrix and tridiagonal matrix? **Ans:**

Sparse Matrix: A sparse matrix is a matrix in which most of the elements are zero, resulting in a data structure that efficiently stores and manipulates only the non-zero elements to conserve memory and processing resources..

0. Characteristic. **0. Triangular Matrix.** **0. Tridiagonal Matrix**

1. Definition. **1. A matrix in which all elements above or below the main diagonal are zero.** **1. A matrix in which all elements outside the main diagonal and its adjacent diagonals (above and below) are zero.**

2. Non-zero Elements. **2. Main diagonal and either above or below it contain non-zero elements.** **2. Main diagonal and its two adjacent diagonals contain non-zero elements; all other elements are zero.**

3. Number of Non-zero Diagonals. **3. One (main diagonal).** **3. Three (main diagonal and its two adjacent diagonals).**

4. Application. **4. Often used for systems with triangular symmetry, such as solving systems of linear equations.**

4. Frequently used in numerical analysis for solving differential equations and finite difference methods.

9. We often store sparse matrices in a 1D array to save spaces. What is the memory saving if we store a sparse matrix in a 1D array rather than a 2D array? Ans:

Storing a sparse matrix in a 1D array can save a significant amount of memory compared to using a 2D array. The exact amount of memory saved depends on the sparsity of the matrix, which is the proportion of zero elements to non-zero elements.

In a 2D array, memory is allocated for every element, whether it's zero or non-zero. In contrast, when using a 1D array to store a sparse matrix, you only need to store the non-zero elements along with their row and column indices. Therefore, the memory saving can be substantial, especially when the matrix is highly sparse and contains mostly zero elements.

The greater the sparsity, the more memory you save by using a 1D array representation.

10. What is pointer array? How a pointer can save memory space when storing a 2D array? Ans: Pointer Array: A pointer array is an array in which each element is a pointer to another data structure or value. In the context of a 2D array, a pointer array is used to efficiently represent a matrix. Instead of allocating memory for the entire 2D array, a pointer array consists of pointers (addresses) to individual rows or arrays, and each row can have a different length.

Using a pointer array can save memory space when storing a 2D array because:

1. It allows you to allocate memory only for the necessary rows, rather than allocating a fixed-size rectangular block for the entire 2D array.

2. Different rows can have different lengths, which is useful for representing irregularly sized matrices or sparse data efficiently.

By dynamically allocating memory for each row based on its actual size, you conserve memory resources, especially when dealing with large and sparsely populated 2D data structures.

11. Is there any differences between a record and a linear array? Ans:

0. Characteristic. 0. Linear Array. 0. Record

1. Structure. 1. One-dimensional, sequence of elements. 1. Composite, stores multiple fields.

2. Elements. 2. Homogeneous (usually of the same data type). 2. Heterogeneous (different data types for each field).

3. Access. 3. By index (position). 3. By field name or label.

4. Use Cases. 4. Lists of similar elements. 4. Representing structured data entities, entities with multiple properties.

5. Example. 5. [1, 2, 3, 4, 5]. 5. {Name: "John", Age: 30, Address: "123 Main St"}

12. How can you locate element

a_{ij} of a sparse matrix from a 1D array?

Ans:

Depending on the ordering (row-major or column-major), you can calculate the index in the 1D array corresponding to element a_{ij} using the formula:

- Row-Major Order: Index = $(i * \text{num_columns}) + j$
- Column-Major Order: Index = $(j * \text{num_rows}) + i$